



NavSight

See the way forward

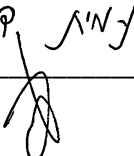
Roey Ben Harush, 315676163,
Tamir Sobuh, 206029084,
Morad Zubidat , 208156828

Supervised by:

Amit Dunsky

SDD SUBMITTED FOR THE DEGREE
"SOFTWARE ENGINEERING B.Sc."

Shenkar College
Faculty of Engineering

Supervisor: אמית דנסקי


Date: 02 / 02 / 2026

Content

1. Introduction	2
a. System Overview	2
b. Purpose	2
c. Scope	2
d. Definitions and Acronyms	2
e. Constraints	2
2. System Architecture	3
a. Architectural Description and Design: Roles, Activities and Data	3
b. The Life Cycle of the System	3
3. Literature Survey	4
a. Problem Survey	4
b. Solution Survey	4
c. Discussion and Conclusion	4
4. Design	5
a. Data Design	5
i. Database Description	5
ii. Global Data Structures Design	5
b. Structural Design	5
i. Class Diagram	5
c. Interactions Design	5
i. Use Cases	5
ii. Sequence Diagram	5
iii. Activity Diagram	5
d. Software Architecture Pattern	5
i. Three-tier: Data, Logic and Presentation tiers	5
ii. MVVM: Model, View, ViewModel structure	5
e. Testing Platform	5
5. Appendix-A	5
a. POC discussion (if relevant)	5
6. Appendix-B	5
a. Team Roles	5
7. Appendix-C	5
a. Schedule / Gantt (possible print screen or sharable link)	5
8. Appendix-D	6
a. System Screens (Updated version after FRS)	6

1. Introduction

a. System Overview

NavSight is a self-contained, real-time mobile navigation system designed for Android devices. It leverages Visual-Inertial Odometry (VIO) combined with Dead Reckoning (DR) to provide continuous position tracking without dependence on Global Navigation Satellite Systems (GNSS). The system is engineered to operate entirely offline, processing sensor data locally on the device to deliver a responsive and private navigation experience. Its primary use case is urban micromobility, enabling users to navigate environments where GPS signals are unreliable or unavailable, such as dense urban canyons, tunnels, and indoor spaces.

b. Purpose

The purpose of the NavSight system is to overcome the limitations of traditional satellite-based navigation in challenging environments. GNSS signals are often blocked, reflected, or degraded in urban canyons created by tall buildings, inside tunnels and underground passages, and within indoor environments. NavSight provides a reliable navigation alternative by using the device's own camera and motion sensors to track movement relative to a starting point, ensuring users can navigate confidently regardless of satellite signal availability.

c. Scope

In-Scope: The project encompasses the development of a Visual Odometry (VO) pipeline for processing camera frames, a Dead Reckoning (DR) module utilizing Inertial Measurement Unit (IMU) data, a sensor fusion algorithm to combine VO and DR estimates, a native Android application with a real-time path visualization UI, and complete offline operation with no network dependency.

Out-of-Scope: The project does not include full 3D Simultaneous Localization and Mapping (SLAM) with loop closure, cloud-based processing or data storage, integration with external map services, turn-by-turn navigation instructions, or voice guidance features.

d. Definitions and Acronyms

VIO (Visual-Inertial Odometry): A technique that estimates device motion by fusing visual information from a camera with inertial measurements from an IMU.

VO (Visual Odometry): The process of determining a device's position and orientation by analyzing sequential camera images.

DR (Dead Reckoning): A method of calculating current position by using a previously known position and advancing it based on estimated speed and direction from inertial sensors.

IMU (Inertial Measurement Unit): A sensor package typically containing accelerometers and gyroscopes that measures linear acceleration and angular velocity.

GNSS (Global Navigation Satellite System): A general term for satellite-based navigation systems, including GPS, GLONASS, Galileo, and BeiDou.

JNI (Java Native Interface): A programming framework that allows Java/Kotlin code running in the Android runtime to call and be called by native applications written in C/C++.

e. Constraints

Hardware Dependency: The system's accuracy and performance are fundamentally limited by the quality of the device's camera and IMU sensors. Lower-quality sensors will result in higher drift and less reliable tracking.

Environmental Limitations: Visual odometry requires sufficient lighting and visual texture in the environment. Performance will degrade in low-light conditions, featureless environments (e.g., blank walls), or with excessive motion blur.

Real-time Processing: All computations must complete within a strict time budget (target <200ms per frame) to provide responsive feedback. This constrains the complexity of algorithms that can be employed.

Power Consumption: Continuous use of the camera and high-frequency IMU sampling are battery-intensive. The system must be optimized to balance accuracy with acceptable battery drain for practical usage durations.

Platform: The application targets Android 10 (API level 29) and above, utilizing the Android NDK for native C++ components.

2. System Architecture

a. **Architectural Description and Design:** Roles, Activities and Data

The NavSight system employs a multi-layered architecture designed for modularity, performance, and clear separation of concerns.

Layer 1 - Presentation Layer (Android UI): Built with Kotlin and Jetpack Compose, this layer is responsible for rendering the camera preview, displaying the tracked path, showing system status (e.g., tracking quality, mode), and handling user interactions (start, stop, pause, resume).

Layer 2: Application Logic Layer (Orchestrator)

Role: System Orchestration & Sensor Management.

Technology: Kotlin, Android SDK (`SensorManager`, `CameraX`).

Responsibilities:

- Manages the application lifecycle and permissions (`CAMERA`, `ACCESS\_FINE\_LOCATION`).
- Initializes and synchronizes hardware sensors (Accelerometer @ 200Hz, Gyroscope @ 200Hz, Camera @ 30fps).
- Acts as the bridge between the UI and the Native Core, passing raw data down and receiving processed `VioData` objects up.

Layer 3: Native Interface Layer (JNI Bridge)

Role: Data Marshalling & Thread Management.

Technology: C++, JNI.

Responsibilities:

Marshals data between the Java Virtual Machine (JVM) and the Native Heap (converting `ByteArray` frame data to `cv::Mat`).

Manages the dedicated background **VIO Thread** to ensure heavy processing never blocks the UI.

Provides thread-safe queues (`frame\_queue`, `accel\_queue`) for asynchronous data ingestion.

Layer 4: Native Processing Core (C++)

Role: Algorithmic Computation (The Engine).

Technology: C++, OpenCV 4.x.

Components:

- **VisionModule:**
- The computer vision powerhouse. It handles feature detection, KLT optical flow tracking, Essential Matrix estimation, and outlier rejection (RANSAC).
- **IMUPreintegrator:** A mathematical module that performs on-manifold integration of raw inertial data. It calculates the delta-position, delta-velocity, and delta-rotation accumulated between two camera frames.
- **SensorFusionEngine:** The logic core that fuses the visual pose with the preintegrated inertial pose. It handles **Automatic Scale Estimation**, **Gravity Alignment**, and the dynamic switching between VIO and Dead Reckoning modes.

The Life Cycle of the System

The system operates through the following states:

- 1. Initialization:** The application launches, requests necessary permissions (camera, sensors), and loads native libraries.
- 2. Calibration:** The system performs an initial sensor warm-up and may prompt the user to perform a brief calibration motion if required.
- 3. Active Tracking (VIO Mode):** The primary operational state. The system fuses camera and IMU data to produce continuous pose estimates. The UI displays the live path.
- 4. Degraded Tracking (DR Mode):** If visual tracking is lost (e.g., camera covered, low light), the system automatically falls back to dead reckoning using only IMU data. A visual indicator alerts the user to the reduced accuracy.
- 5. Recovery:** When visual features become available again, the system transitions back to full VIO mode.
- 6. Paused:** The user can pause the session, suspending all sensor processing while retaining the current path data.
- 7. Termination:** The user stops the session. A summary of the tracked path may be displayed, and resources are released.

Literature Survey

b. Problem Survey

Visual-Inertial Odometry (VIO) addresses the fundamental challenge of estimating a device's position and orientation using only onboard sensors, without relying on external infrastructure like GPS satellites. This problem is critical for navigation in GNSS-denied environments such as urban canyons, indoor spaces, tunnels, and underground facilities.

The core technical challenge lies in fusing two complementary but imperfect sensor modalities. Cameras provide rich geometric information through feature tracking but suffer from scale ambiguity, motion blur, and failure in low-texture or low-light conditions. Inertial Measurement Units (IMUs) offer high-frequency motion estimates immune to visual conditions but accumulate drift rapidly when double-integrated.

Key approaches in the literature include feature-based SLAM systems like ORB-SLAM that extract and match distinctive visual features across frames, direct methods such as LSD-SLAM that operate on image intensities, Visual Odometry approaches focusing on incremental motion estimation, and IMU pre-integration techniques for efficient sensor fusion.

c. Solution Survey

The NavSight system addresses the VIO problem by implementing a tightly-coupled sensor fusion approach that combines visual feature tracking with inertial pre-integration.

Sensor Fusion Strategy: The system employs a tightly-coupled filtering framework that combines visual pose estimates with pre-integrated IMU measurements. When visual tracking quality is high, camera-based motion estimates are prioritized for accurate relative pose estimation, while inertial data is used to stabilize orientation and resolve metric scale. In cases of disagreement between the visual and inertial estimates, the fusion module weights each sensor modality based on its confidence level (e.g., feature tracking quality and IMU signal stability). If visual tracking becomes unreliable or unavailable (e.g., low-light or textureless environments), the system automatically degrades to inertial-only dead reckoning mode, ensuring continuous motion estimation until visual data recovers.

Visual Odometry Pipeline: Based on principles from established VO systems, our implementation uses sparse feature detection and KLT optical flow for inter-frame tracking, providing relative pose estimates from camera motion.

Inertial Pre-integration: Following established methodology, we pre-integrate

accelerometer and gyroscope measurements between visual keyframes, efficiently handling high-frequency IMU data (100-200Hz).

Sensor Fusion Strategy: We employ a filtering approach that fuses visual pose estimates with pre-integrated IMU measurements, prioritizing real-time performance on mobile hardware.

Dead Reckoning Fallback: When visual tracking fails, the system automatically transitions to dead reckoning mode using only inertial data, providing continuity until visual tracking recovers.

d. Discussion and Conclusion

The literature review reveals a clear trade-off between accuracy and computational efficiency in VIO systems. Full SLAM solutions offer superior accuracy through loop closure and global bundle adjustment, but their computational demands make them challenging for mobile devices.

Our approach draws inspiration from lightweight VIO systems that prioritize real-time performance. By avoiding expensive operations like loop closure, we maintain the latency required for responsive navigation feedback (<200ms target). The trade-off is that positional drift will accumulate over time.

The dead reckoning fallback mechanism is crucial for practical deployment in urban environments that frequently present visual challenges. Future work could explore integration with occasional GNSS fixes to bound accumulated drift.

3. Design

a. Data Design

i. Database Description

The NavSight system operates as a real-time, offline processing engine and does not utilize a traditional persistent relational database (SQL) for its core navigation loop.

Transient Sensor Storage: High-frequency data from the camera (30 FPS) and IMU (200Hz) is buffered in circular buffers within the Native Memory (RAM) to minimize latency. This data is discarded once processed.

Session Persistence: If required for post-session analysis, trajectory data (waypoints) can be serialized to local JSON/CSV files, but this is optional and not required for real-time operation.

Configuration Management: Application settings (e.g., permissions, user preferences) are managed via Android `SharedPreferences`

Global Data Structures Design

The system relies on optimized data structures defined in the Native C++ layer to handle the intense data throughput required for VIO.

1. Sensor Frame Packet (`CamFrame`)

Represents the fundamental unit of visual input.

jlong timestamp: Nanosecond-precision timestamp synchronized with the Android SystemClock.elapsedRealtimeNanos().

cv::Mat yuvMat: Raw image data in YUV_420_888 format. This is efficiently converted to Grayscale for computer vision processing.

int width, height: Dimensions of the video feed.

2. Inertial Data Packets (`AccelData`, `GyroData`)

Structures for buffering high-frequency inertial measurements.

long timestamp_ns: Timestamp of the sensor event.

float x, y, z: 3-axis acceleration (m/s^2) or angular velocity (rad/s).

3. Vision Output Packet (`VisionOutput`)

The result of the Visual Odometry pipeline for a single frame.

cv::Mat rotation: A 3x3 rotation matrix representing the device's relative rotation since the previous frame.

cv::Mat translation: A 3x1 translation vector representing relative movement.

float tracking_quality: A confidence score (0.0 - 1.0) derived from the ratio of feature inliers.

bool is_valid: A flag indicating if the visual tracking was successful. `False` triggers Dead Reckoning fallback.

4. VIO Data Transfer Object (`VioData`)

The bridge object used to transport the final fused pose from the C++ engine to the Kotlin UI layer.

double x, y, z: Global position in meters (relative to start).

double roll, pitch, yaw: Global device orientation.

float[] trackedPoints: Array of 2D coordinates for visualizing tracked features on the UI.

double estimatedScale: The real-time computed scale factor, allowing conversion from visual units to meters.

b. Component Design

The system architecture is centered around the **VIO Engine Layer**, implemented in C++ for performance.

i. Vision Processing Module (Visual Odometry)

This module is responsible for estimating device motion from camera images. It utilizes a feature-based approach:

1. Feature Detection: Uses the **Shi-Tomasi** corner detector

(`cv::goodFeaturesToTrack`) to identify up to 200 distinct, trackable points in the image.

- 2. Optical Flow:** Implements the **Lucas-Kanade (LK)** Pyramidal method (``cv::calcOpticalFlowPyrLK``) to track these features from the previous frame to the current frame.
- 3. Outlier Rejection:** Applies the **RANSAC** algorithm with an Essential Matrix model (``cv::findEssentialMat``).
- 4. Configuration:** A high RANSAC confidence level (99.99%) is used together with a sub-pixel reprojection threshold to reject outliers caused by dynamic objects and sensor noise.

The threshold is applied in floating-point image coordinates, allowing sub-pixel accuracy supported by the Lucas-Kanade optical flow implementation. The selected value represents a trade-off between robustness to noise and sensitivity to small camera motions.

- 5. Pose Recovery:** Decomposes the Essential Matrix into rotation (R) and translation (t) components.

ii. Automatic Scale Estimation & Gravity Alignment

A critical innovation in NavSight is solving the "Monocular Scale Ambiguity" problem (where a single camera cannot tell the difference between 1 meter and 10 meters).

Gravity Initialization: At startup, the system collects 20+ accelerometer samples to determine the gravity vector (g). This establishes the "Down" direction and aligns the coordinate system with the real world.

Scale Fusion: The system continuously compares the "visual distance" (in arbitrary units) with the "inertial distance" (in meters) derived from double-integrating accelerometer data. This allows the system to output real-world coordinates without requiring pre-calibration or stereo cameras.

iii. Sensor Fusion & Dead Reckoning (DR)

The Fusion Engine combines the outputs of the Visual pipeline and the IMU:

VIO Mode (Primary): When visual quality is high, the system fuses the precise relative motion from the camera with the rotation from the gyroscope. The accelerometer is used to correct scale.

Dead Reckoning Mode (Fallback):

If visual tracking fails (e.g., ``VisionOutput.is_valid == false`` due to low light or untextured walls), the system automatically degrades to DR mode. In this state, position is propagated solely by integrating IMU acceleration and velocity. This ensures navigation continuity until visual features are recovered.

Interface Design (JNI Bridge)

The Native Interface Layer facilitates high-speed communication between the Android Runtime (ART) and the Native code. It exposes the following API:

startVIO() / stopVIO(): Lifecycle management for the background processing thread.

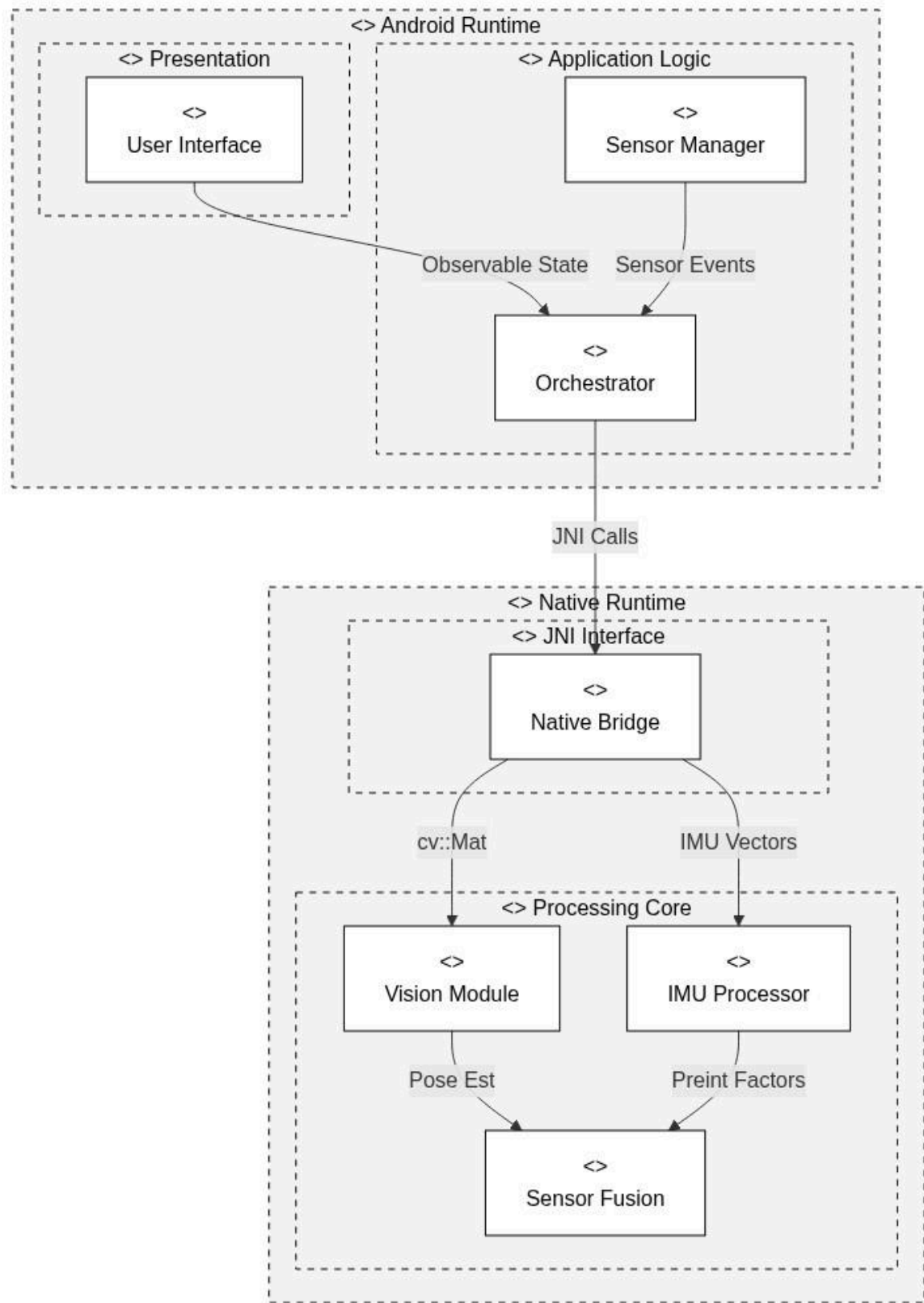
processCameraFrame(byte[] data, ...): Asynchronous entry point for video frames. Returns the **latest available** global pose immediately to ensure the UI remains responsive (60 FPS) even if processing takes longer

processAccelerometer(x,y,z) / processGyroscope(x,y,z): High-frequency hooks for feeding the inertial buffers.

resetVIO(): Zeros the global pose and re-initializes the gravity alignment vector

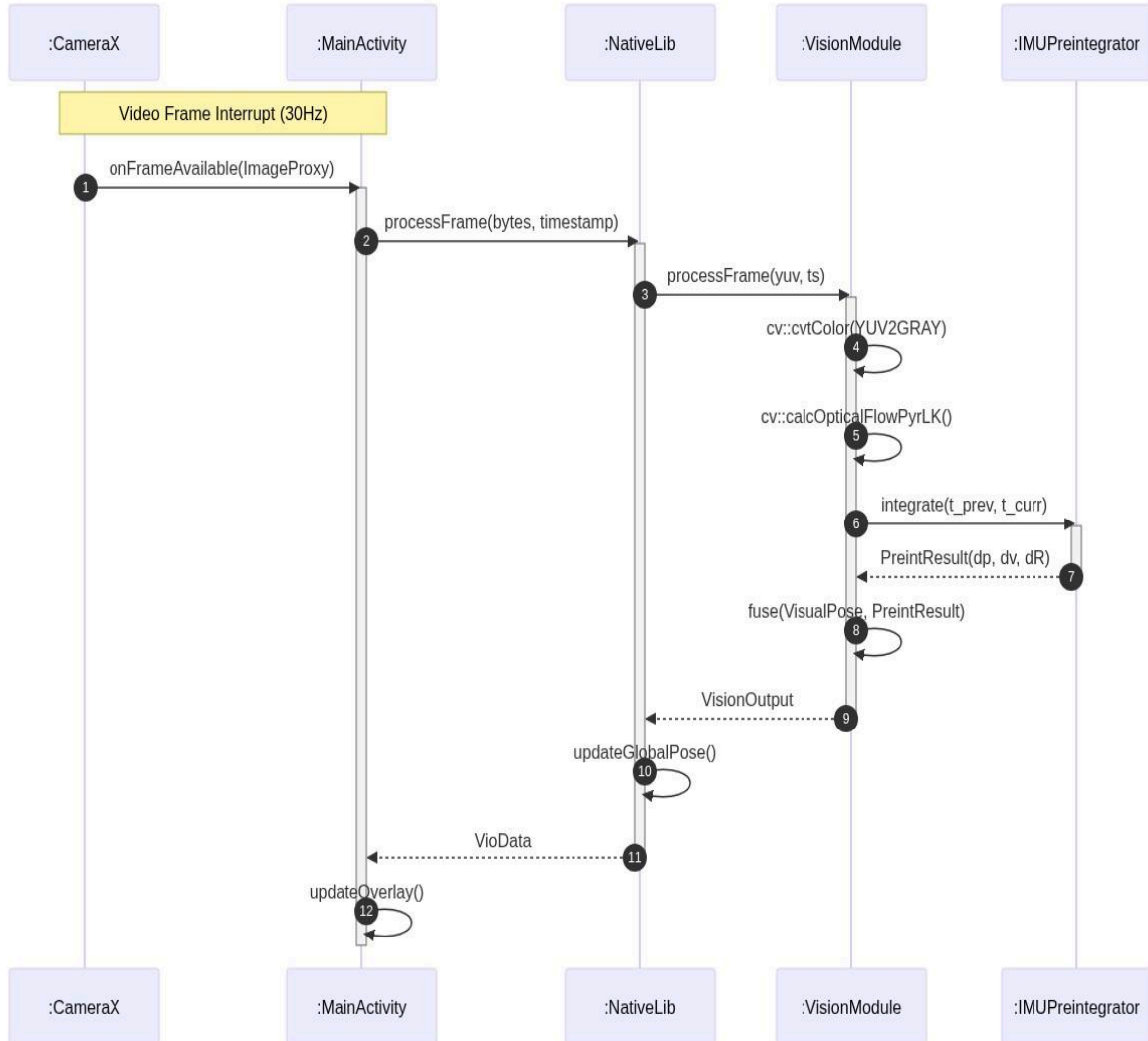
Structural Design

i. System Architecture Diagram



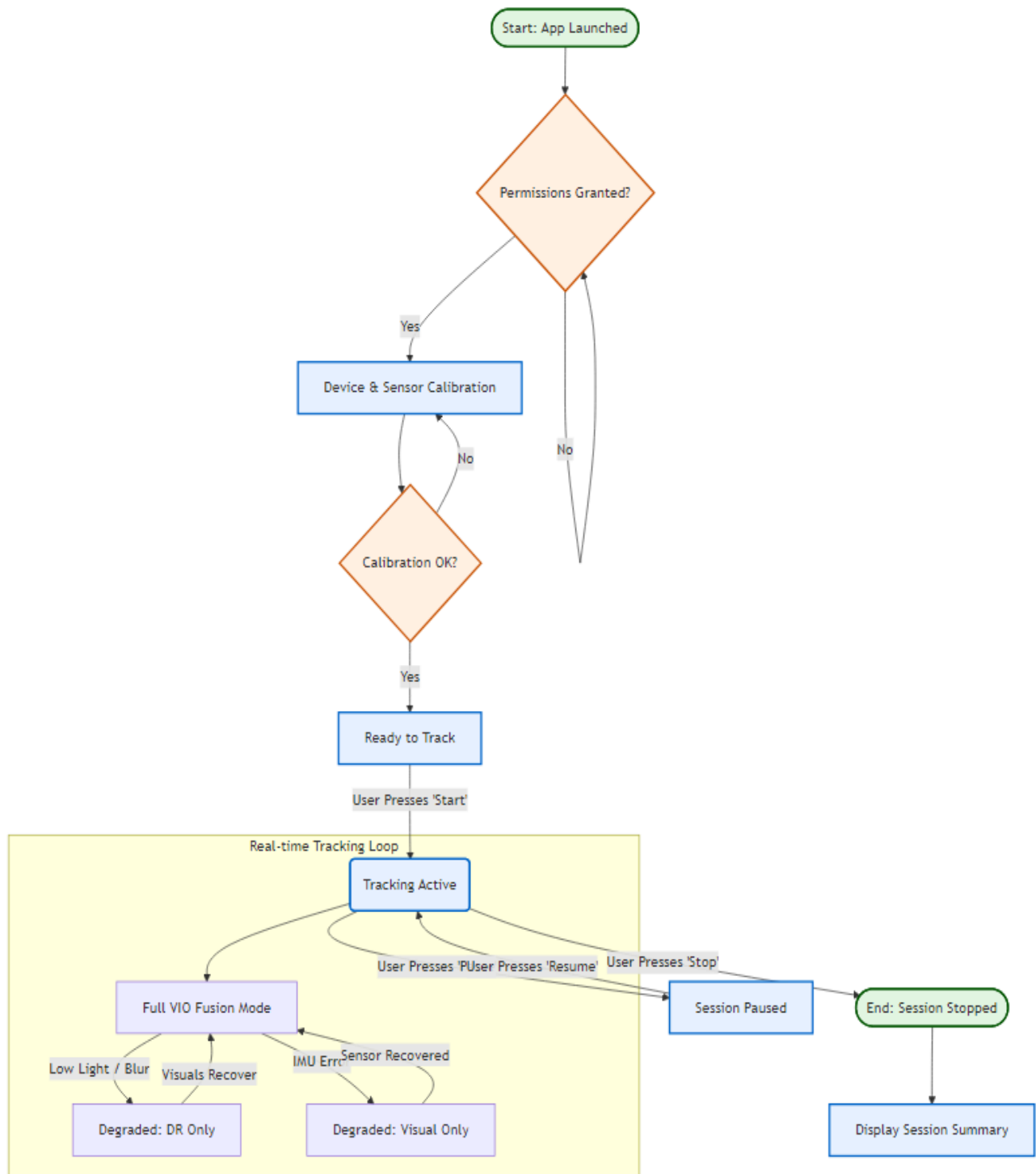
ii. Sequence Diagram

This diagram shows the sequence of interactions for processing a single sensor frame packet, from user input through the Android UI, Application Logic, and Native Core layers.

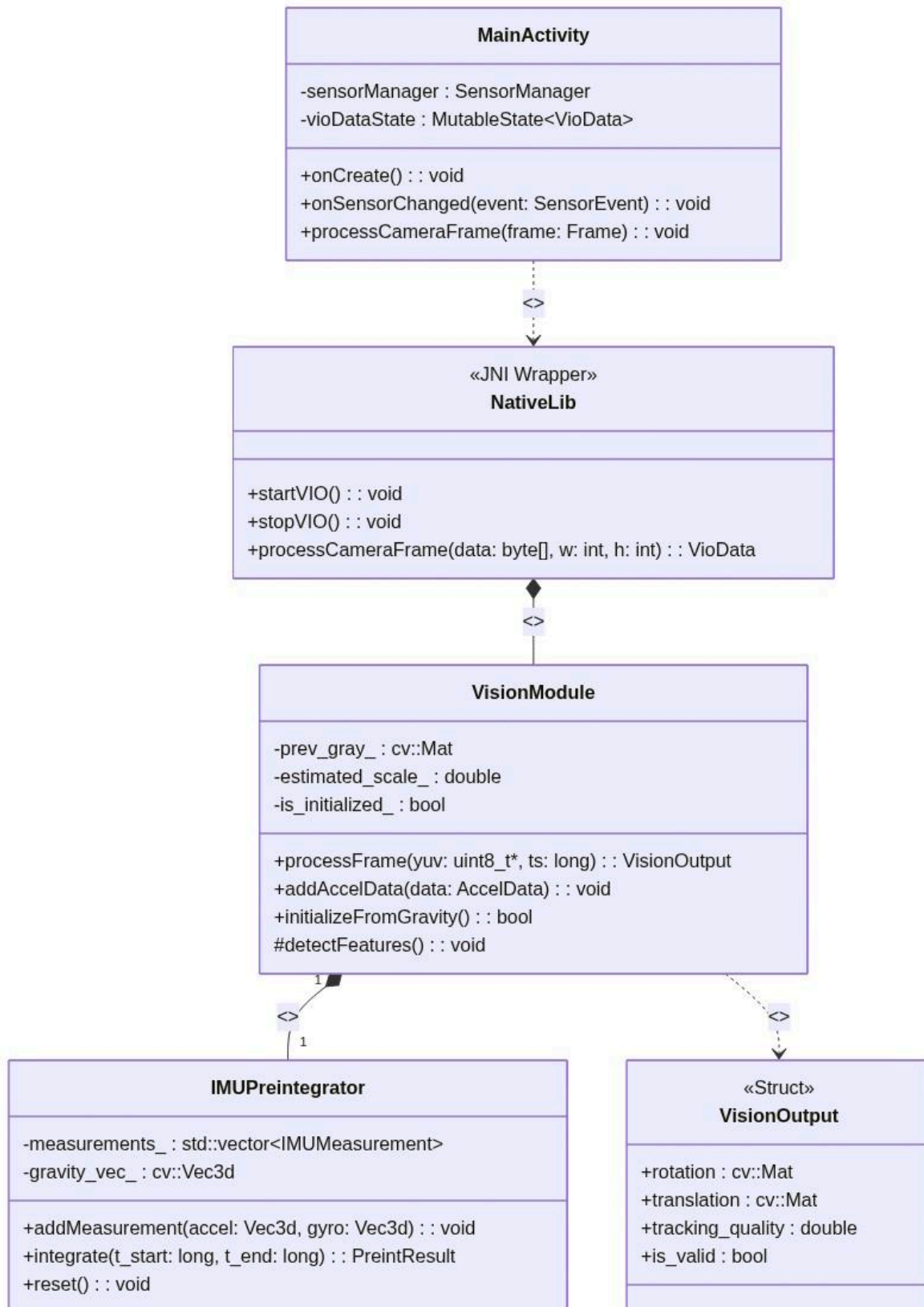


ii. Activity Diagram

This diagram models the system's operational states and transitions, reflecting the lifecycle from app launch through permissions, calibration, active tracking, degraded modes, and session termination.



iii. Class Diagram :



c. Interactions Design

i. Use Cases

UC-1: Track a Navigation Path - The user initiates the application, calibrates the device, starts tracking, navigates their route with real-time path display, and stops to view a summary.

UC-2: Handle Degraded Visuals - During tracking, the user enters a poorly lit area. The system detects feature loss, switches to DR mode, notifies the user, and automatically resumes VIO when conditions improve.

UC-3: Pause and Resume - The user pauses the session mid-route, the system suspends processing, and later resumes tracking from the last known position.

Algorithm Design

.This section details the mathematical and logical operations of the core components

i. Feature Detection: Shi-Tomasi

To track camera motion, the system must first identify "features"—points in the image that are easy to track (typically corners). We use the **Shi-Tomasi** corner detector, which is an improvement on the Harris detector

- **Logic:** For every pixel (u, v) , we compute the gradient matrix
- **Criterion:** A pixel is a corner if both eigenvalues are above a threshold.
Shi-Tomasi simplifies this by checking if $\min(\lambda_1, \lambda_2) > \lambda_{\text{thresh}}$.
- **Optimization:** We limit detection to 200 features and enforce a minimum Euclidean distance between points to ensure a uniform distribution across the image.

ii. Feature Tracking: Lucas-Kanade Optical Flow

Once features are detected in Frame N-1, we find their new positions in Frame N using

.Lucas-Kanade (LK) Optical Flow

- **Assumption:** The brightness of a pixel does not change between frames ($I(x,y,t) = I(x+dx, y+dy, t+dt)$), and local neighbors move together.
- **Pyramidal Implementation:** To handle large motions, we use a Gaussian Image Pyramid. Tracking starts at the lowest resolution (coarse level) to handle large displacements and refines the position at higher resolutions (fine levels).
- **Output:** A set of point correspondences $\{p_{n-1}, p_n\}$ linking the two frames.

iii. Pose Estimation: Essential Matrix & RANSAC

.Given the set of point correspondences, we estimate the camera's motion

Given the set of point correspondences between consecutive frames, the camera motion is estimated by computing the Essential Matrix using a RANSAC-based robust estimation process. The Essential Matrix encodes the relative rotation and translation direction between the two .camera poses

- **Epipolar Constraint:** For a valid motion, points must satisfy $p_n^T E p_{n-1} = 0$, where E is the Essential Matrix.
- **RANSAC (Random Sample Consensus):** Real-world tracking contains outliers (moving objects, bad matches). RANSAC iteratively selects 5 random points, computes a hypothesis E , and checks how many other points fit this model (inliers). The model with the most inliers is chosen.
- **Decomposition:** The best E is decomposed using Singular Value Decomposition (SVD) into the Rotation Matrix (R) and Translation Vector (t). Note that t is normalized (scale = 1).

iv. Inertial Preintegration

To process high-frequency IMU data efficiently without re-integrating it every time the vision .pose is updated, we use **Manifold Preintegration**

- **Concept:** Instead of integrating position/velocity in the global frame (which changes as the pose estimate changes), we integrate motion *relative* to the frame k .
- **Rotation:** We accumulate gyroscope readings on the $SO(3)$ manifold:
- **Velocity & Position:** We sum accelerometer readings, rotating them by the accumulated ΔR_{ij} to keep them in the local frame.

$$\Delta v_{ij} = \sum_{k=i}^{j-1} \Delta R_{ik} a_k \Delta t$$

$$\Delta p_{ij} = \sum_{k=i}^{j-1} (\Delta v_{ik} \Delta t + \frac{1}{2} \Delta R_{ik} a_k \Delta t^2)$$

- **Result:** This gives us a rigorous measurement of "how much the device moved" purely from IMU data, which is then used to constrain the visual estimation.

v. Automatic Scale Estimation

To solve the monocular scale ambiguity (where a 1m move looks like a 10m move at 10x .distance), we compare Visual and Inertial measurements

- **Visual Displacement:** $d_{vis} = ||t_{vis}||$ (Unitless)
- **Inertial Displacement:** $d_{imu} = ||\Delta p_{imu}||$ (Meters)
- **Scale Factor:** $s = d_{imu} / d_{vis}$
- **Filtering:** The raw scale factor is noisy. We use an exponential moving average (EMA) filter ($s_{new} = \alpha \cdot s_{raw} + (1-\alpha) \cdot s_{prev}$) to smooth the estimate over time. The EMA update requires constant time complexity $O(1)$ per frame and minimal memory usage. This design choice ensures real-time performance on smartphone hardware and avoids introducing significant computational overhead into the VIO pipeline.

Software Architecture Pattern

i. Four-tier Architecture: Presentation, Application Logic, Native Interface and Processing Core

The NavSight system employs a layered architecture with four primary tiers: Presentation Layer (Android UI), Application Logic Layer, Native Interface Layer (JNI Bridge), and Native Processing Core (C++). This design ensures modularity, facilitates concurrent processing, and isolates performance-critical native code from the application framework.

ii. MVVM: Model, View, ViewModel structure

Within the Presentation Layer, the application employs MVVM (Model-View-ViewModel) patterns common in modern Android development. The Model consists of data classes like VioData. The View comprises Jetpack Compose UI elements. The ViewModel role is served by MainActivity and associated state management logic.

Testing Platform

The testing strategy encompasses multiple levels:

1. Unit Testing: Native C++ modules tested using GoogleTest framework to validate core algorithm correctness.

2. Integration Testing: JNI boundary tested for correct data marshalling, thread safety, and native object lifecycle management.

3. Application-level Testing: Android application tested using Espresso for UI interaction and JUnit for application logic.

Appendix-A

a. POC discussion

The Proof of Concept validated core NavSight functionality.

POC Objective: Demonstrate accurate movement tracking when the device is held parallel to the ground (90 degrees), displaying trajectory on screen in real-time.

Test Scenario: Device held horizontally while walking forward, backward, left, and right. The system interprets camera feed and IMU data to calculate relative movement.

Success Criteria: Correct direction detection, real-time path updates, tracking continuity during normal walking.

Results: The POC successfully demonstrated VIO tracking with horizontal phone orientation. On-screen visualization correctly reflected movement direction, validating the implementation feasibility.

Appendix-B

a. Team Roles

Roey Ben Harush:

- Lead Android application development (Kotlin, Jetpack Compose)
- System architecture design and layer separation
- JNI bridge implementation
- Task coordination and integration
- UI/UX design

Tamir Sobuh:

- C++ native processing core development
- Vision Processing Module implementation
- Native code optimization
- Unit testing for native components

Morad Zubidat:

- Inertial Pre-integration Module
- Sensor Fusion algorithm
- Sensor synchronization and calibration
- Integration testing and documentation